

Arbitrary Precision Arithmetic in Java

CS1023 - Software Development Fundamentals Project Report

Devarapalli Rupesh
cs24btech11023

May 2025

Contents

1	Introduction	2
2	Project Structure	2
2.1	Packages and Classes	2
2.2	Folder Structure	2
3	Design Overview	2
3.1	AInteger	2
3.2	AFloat	2
3.3	Command Line Interface	2
4	Sample Outputs	3
5	Build and Run Instructions	3
5.1	Using Ant	3
5.2	Using Python Script	3
6	Learnings	3
7	Limitations	3

1 Introduction

This project implements an infinite/arbitrary-precision arithmetic library in Java. It supports all four basic arithmetic operations for both integers and floating point numbers, without relying on Java's built-in 'BigInteger' or 'BigDecimal'.

The goal was to simulate manual operations (addition, subtraction, multiplication, division) using string-based digit manipulation to preserve full precision, especially for very large numbers.

2 Project Structure

2.1 Packages and Classes

- **arbitraryarithmetic.AInteger** – Implements arbitrary-precision signed integers.
- **arbitraryarithmetic.AFloat** – Implements arbitrary-precision floating point using scaled integers.
- **MyInfArith** – Command-line interface to perform arithmetic operations.

2.2 Folder Structure

```
SDF_final|
src/
  main/java/
    arbitraryarithmetic/
      AInteger.java
      AFloat.java
      MyInfArith.java
out/
script2.py
build.xml
```

3 Design Overview

3.1 AInteger

AInteger uses strings to represent large integers and implements arithmetic operations by processing digits manually. Signs are tracked with a boolean flag.

- Internal format: `String` value, `boolean isNeg`
- Operations: `add()`, `subtract()`, `multiply()`, `divide()`

3.2 AFloat

AFloat stores a scaled version of the number using **AInteger**, where decimal digits are tracked via an integer scale. Arithmetic is performed by adjusting scales to a common base.

3.3 Command Line Interface

MyInfArith parses command-line arguments like:

```
java MyInfArith int add 123456 7890
java MyInfArith float div 244727.15202 75964.3891
```

4 Sample Outputs

Integer Addition

```
java MyInfArith int add 23650078224912949497310933240250 42939783262467113798386384401498
Output: 66589861487380063295697317641748
```

Float Division

```
java MyInfArith float div 244727.15202 75964.3891
Output: 3.22160363453775211100855150561
```

5 Build and Run Instructions

5.1 Using Ant

```
ant clean
ant jar
java -cp out MyInfArith int add 123 456
```

5.2 Using Python Script

```
python src/script2.py int sub 123456789 98765
```

6 Learnings

- Implementing arithmetic from scratch builds a deep understanding of digit-level logic.
- Managing large numbers without built-in types requires careful handling of strings and scale.
- Automating builds and scripting Java CLI execution improves development flow.

7 Limitations

- Float division limited to 30 decimal digits as required.
- No support for full expression evaluation or parenthesis.
- No advanced error recovery for malformed input (assumes input is valid).